

Progress Update — 14 April 2026

Project: OptiTrade — financial trading dashboard (dynamic canvas, widgets, AI-assisted insights)

1.0 Introduction

Overall, the team has concentrated on **solution design and implementation** for a web-based trading workspace: a **dynamic widget canvas** (customisable layout), **specialised widgets** (charts, portfolio, news), **brokerage-oriented configuration** (IBKR Flex Web Service), and a **context-aware AI chat** path (Dify integration, with LangChain comparison planned). This update summarises completed work, preliminary UI/results, risks and mitigations, and the near-term development plan.

2.0 Work completed

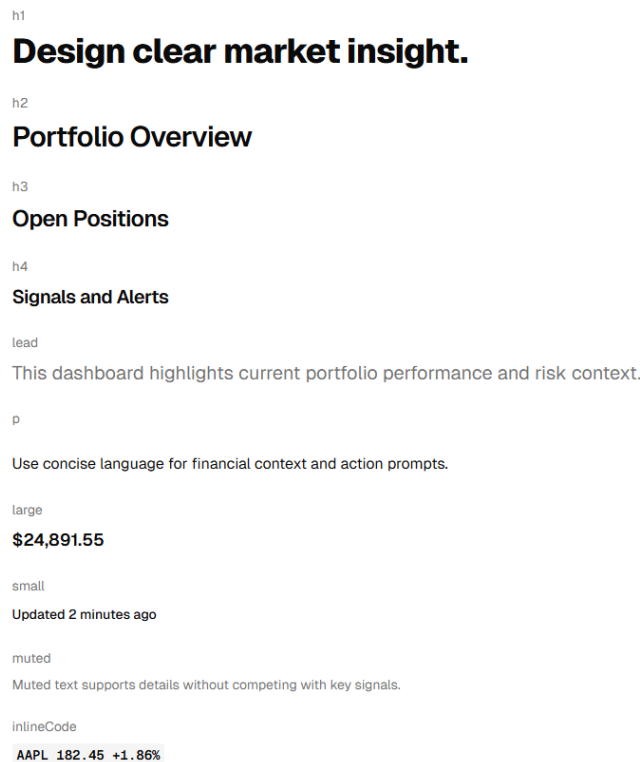
Deliverable	Status	Completed / notes
Solution design — dynamic canvas, widget model, and UX patterns	Completed	Core layout, edit mode, grid/collision behaviour defined and implemented in prototype
Chart widget (candlesticks + indicators)	Completed	TradingView Lightweight Charts integration; timeframe/interval controls
Portfolio widget (multi-size layouts)	Completed	Small / medium / large grid modes; mock data and calculation flows
News widget (aggregation UI, sentiment, risk tags)	Completed	Mock APIs; list/card views; simulated periodic AI updates
AI chat (initial)	In progress	Dify chatbot embedded; bubble toggle; context from widgets planned
Project web page	Completed	Public landing/demo page published (see Section 3.7)

Adjust dates in the table above to match your official milestones if your supervisor requires exact calendar dates.

3.0 Preliminary results

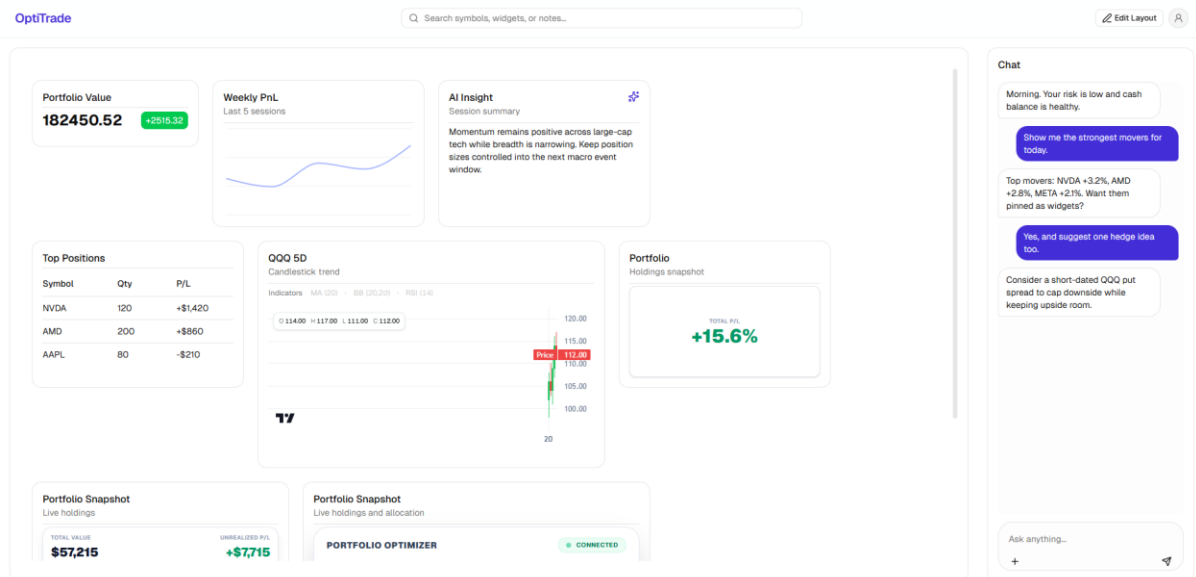
3.1 Design principles

The web application follows a **shadcn/ui**-based theme (**Radix** primitives, **neutral** base palette, **Tailwind CSS**), chosen for accessibility, customisation, and a modern, professional look suited to a financial product. A **primary purple** accent supports a “wealth / clarity / focus” brand direction while keeping the interface calm and readable.



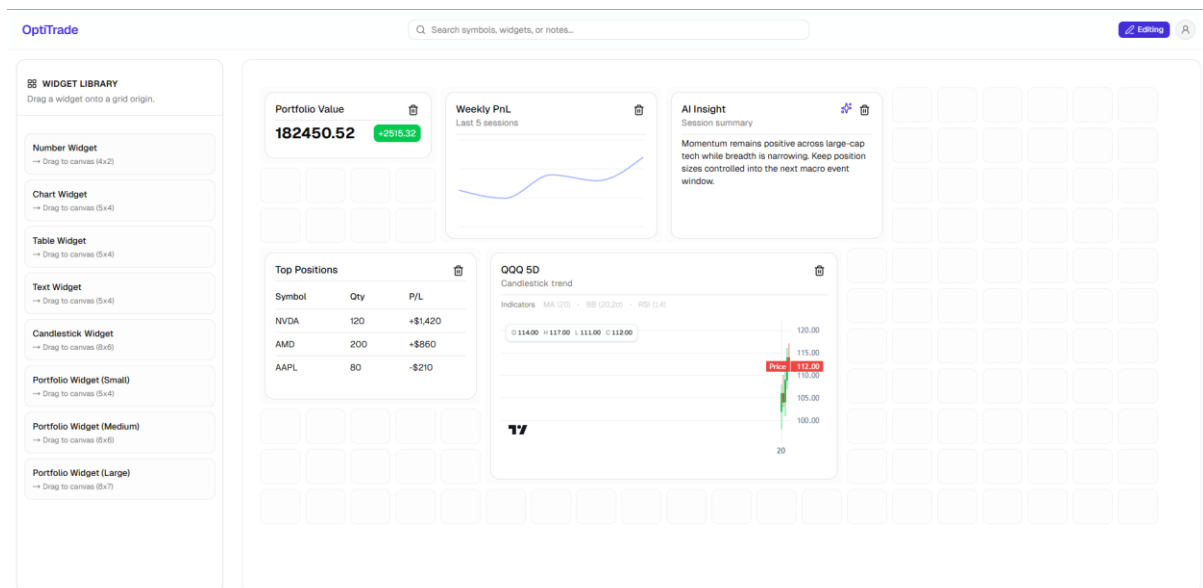
3.2 Dynamic canvas

The main screen is organised into three coordinated areas: a **header** (including normal vs **edit mode**), a central **widget canvas**, and a **widget library** (drawer on desktop, overlay on mobile). The canvas uses a **responsive CSS grid**: widgets span one or more cells; a **ResizeObserver** reflows columns as the viewport changes. In edit mode, users **drag**, **add**, and **remove** widgets; **collision handling** and **drop targets** keep layouts orderly. The result is a flexible personal workspace rather than a fixed dashboard.



The heart of the page is the responsive widget canvas, built on a smart CSS grid system. Each widget occupies one or more grid cells, with compact widgets (like numeric values) taking less space and larger ones (such as candlestick charts or portfolio overviews) spanning more cells for better readability. The grid automatically adjusts to the available screen width at runtime, calculating the right number of columns and using a `ResizeObserver` to reflow the layout whenever the window size changes. This ensures the dashboard always looks balanced and well-organized, without awkward gaps or clipping, regardless of device or screen orientation.

What makes the canvas truly powerful is its free customization. Users can easily enter edit mode to drag widgets around, add new ones from the library, or remove unwanted ones. The system prevents overlaps through intelligent collision detection and automatically adjusts placements when the screen size changes, keeping everything visible and properly arranged. Empty cells appear as clear drop targets during editing, making rearrangement intuitive. As a result, the dashboard feels like a flexible personal workspace rather than a static page—users can freely build and refine their ideal trading view while the underlying grid maintains structure and order.



3.3 Price and technical indicator chart widget

A **candlestick** widget is built on **TradingView Lightweight Charts** for performance and a familiar trading aesthetic.

Features

- **Candlesticks** — OHLC data with colour-coded bodies (e.g. bullish/bearish) and clear borders.
- **Timeframe and interval** — Dropdowns (e.g. 1M, 5D; 1d, 1h).
- **Technical indicators** (toggleable):
 - **MA (20)** — 20-period moving average on price.
 - **BB (20, 2σ)** — Bollinger Bands (upper, basis, lower).
 - **RSI (14)** — RSI in a sub-panel.
- **Live price line** — Dashed horizontal line with axis label.
- **OHLC tooltip** — Hover shows OHLC and active indicator values.

Widget sizing on the grid

Size	Grid dimensions
Small	4 × 6
Large	6 × 12

AAPL Candlesticks



Click the sparkles to show or hide the AI-style summary

Timeframe 1M Interval 1d

Indicators MA (20) · BB (20,2σ) · RSI (14)



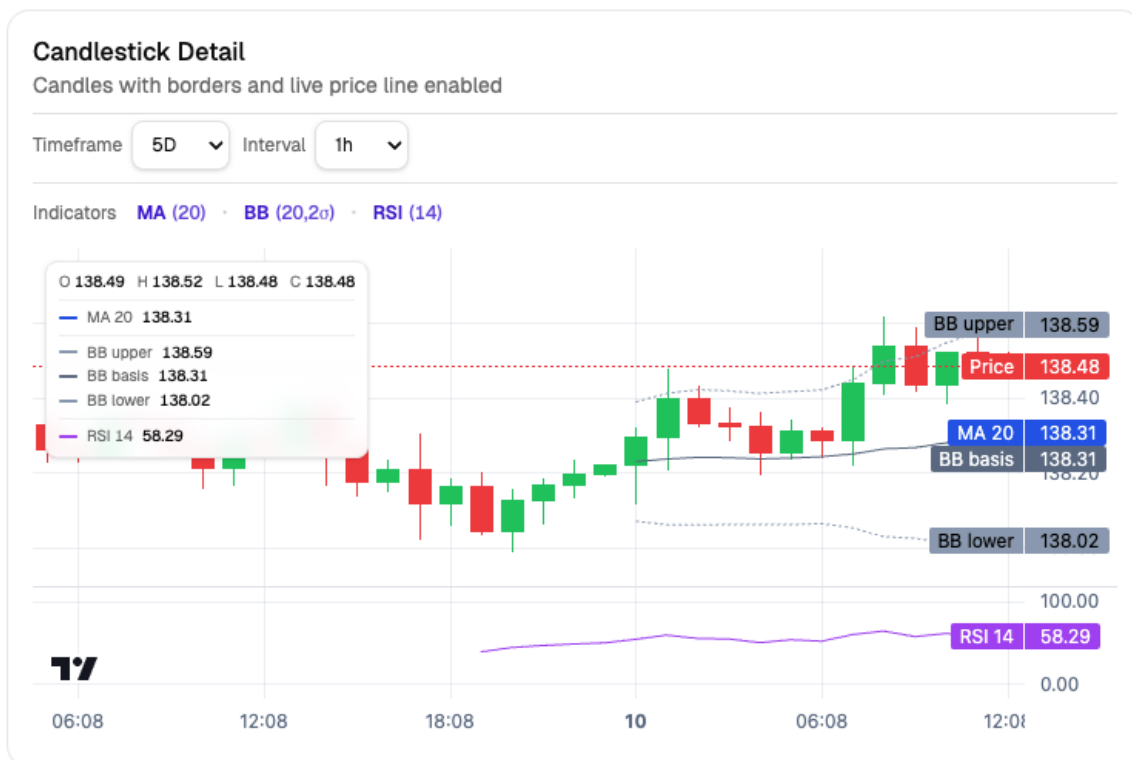
Candlestick Detail

Candles with borders and live price line enabled

Timeframe 5D Interval 1h

Indicators MA (20) · BB (20,2σ) · RSI (14)





The chart reflows within the allocated grid cells.

3.4 Portfolio widget

The widget summarises **allocation**, **P/L**, and related metrics, with a path toward **IBKR** connectivity.

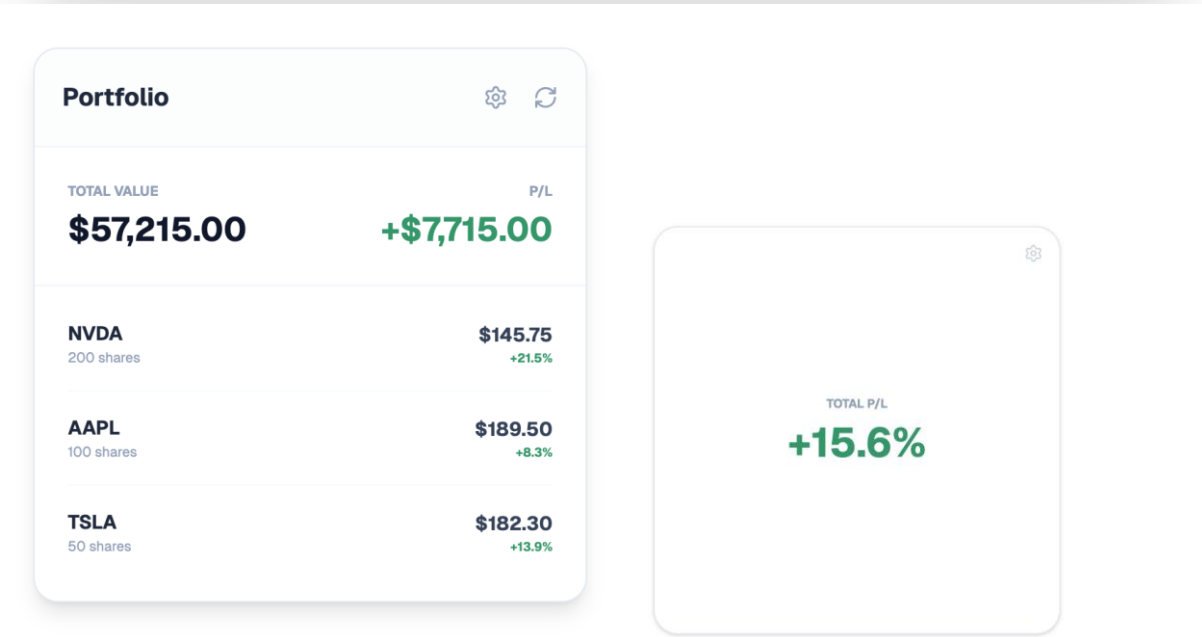
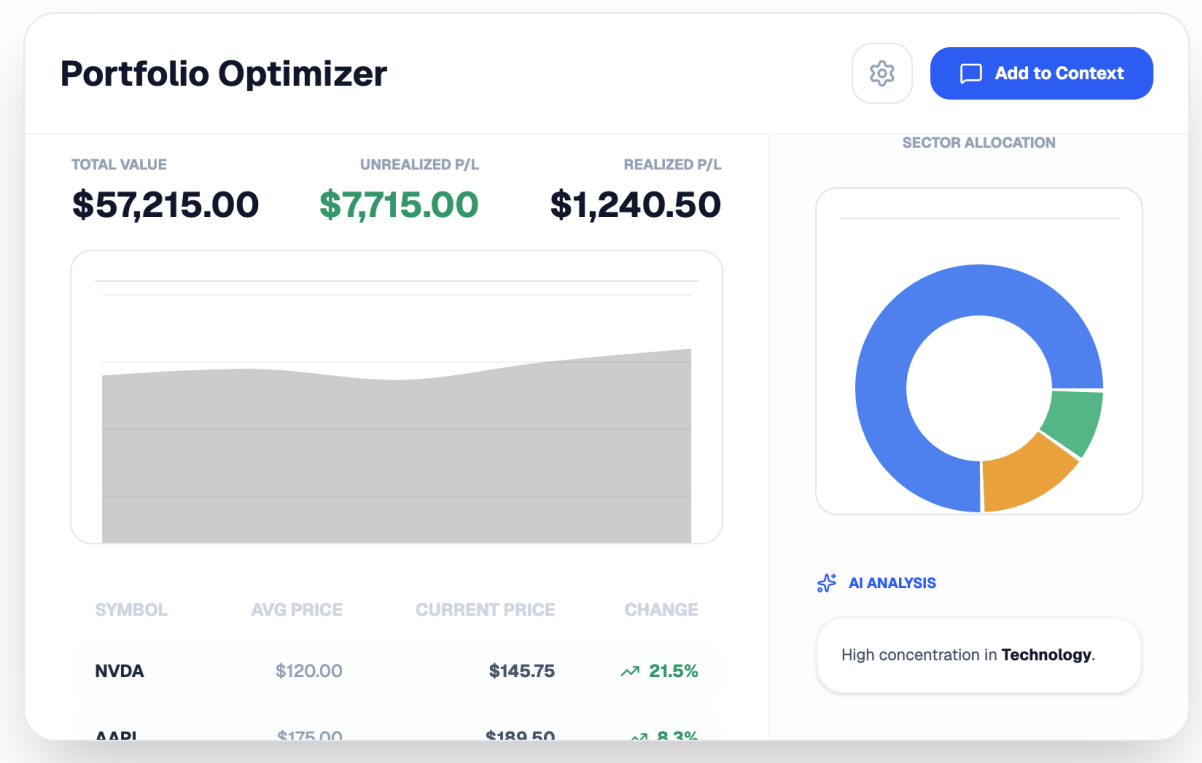
Completed UI features

- **Multi-dimensional layouts** — Small (4×6) P/L snapshot; medium (scrollable positions + key metrics); large (6×12) with charts, tables, and **AI analysis** sidebar.
- **Financial calculations (mock)** — Total value, unrealised/realised P/L, per-position % change (e.g. vs average cost).
- **Sector view** — Pie chart (e.g. Technology, Energy, Cash); **AI-style** concentration notes (e.g. high technology weight).
- **Performance** — Area chart (e.g. five-day trend) with purple-accent styling.
- **Positions table** — shadow-style table; trend icons and colour states.
- **IBKR Flex Web Service panel** — Query ID / token inputs, connect workflow, sync feedback (configuration UI).
- **“Add to context”** — Feeds portfolio state toward the AI chat sidebar (behaviour to be deepened).

Next steps

- Replace **MOCK_STOCKS** with **live** feeds (e.g. Yahoo Finance / FUTU API); **WebSocket** or **polling** for ``currentPrice``.
- **LLM-generated** risk commentary from sector weights and news (beyond static strings).
- **Performance** — Horizons (1D, 1W, 1M, YTD); benchmark vs index (e.g. S&P 500).

- **Persistence and security** — Secure storage for IBKR credentials; persist edit-mode widget sizes on the canvas.



3.5 News widget

Current status

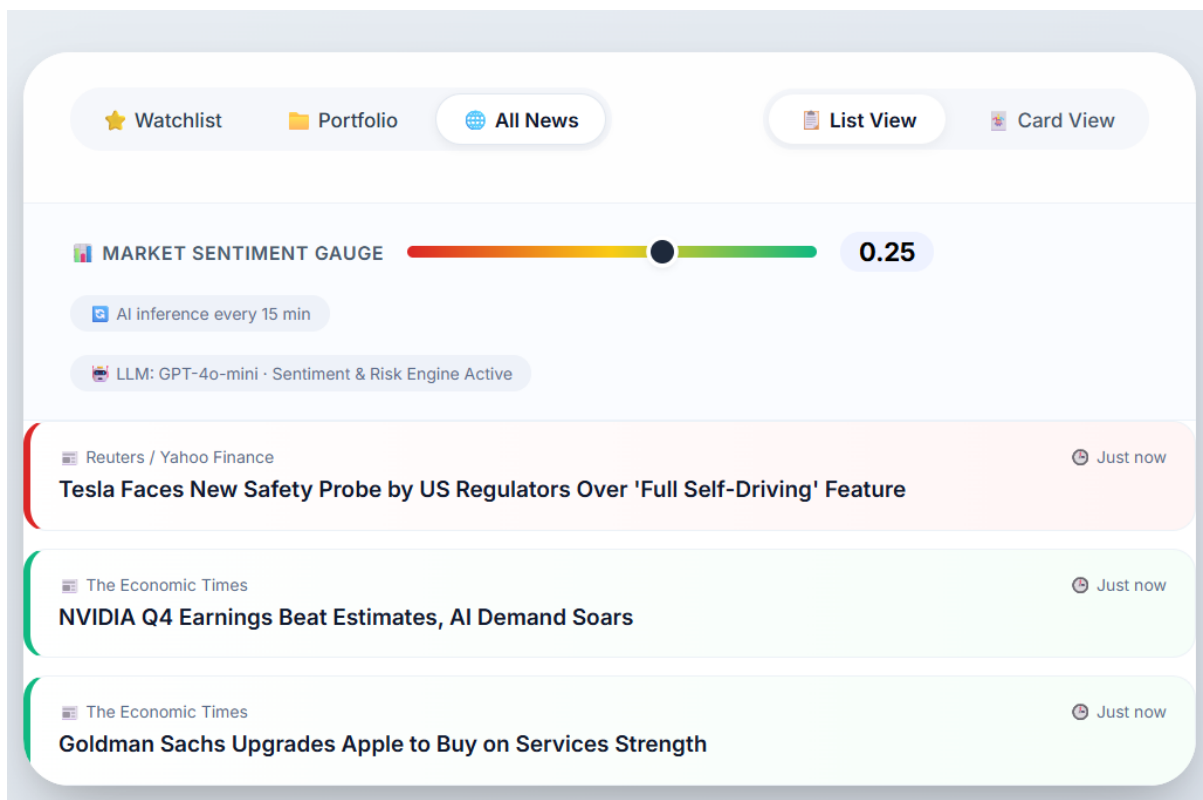
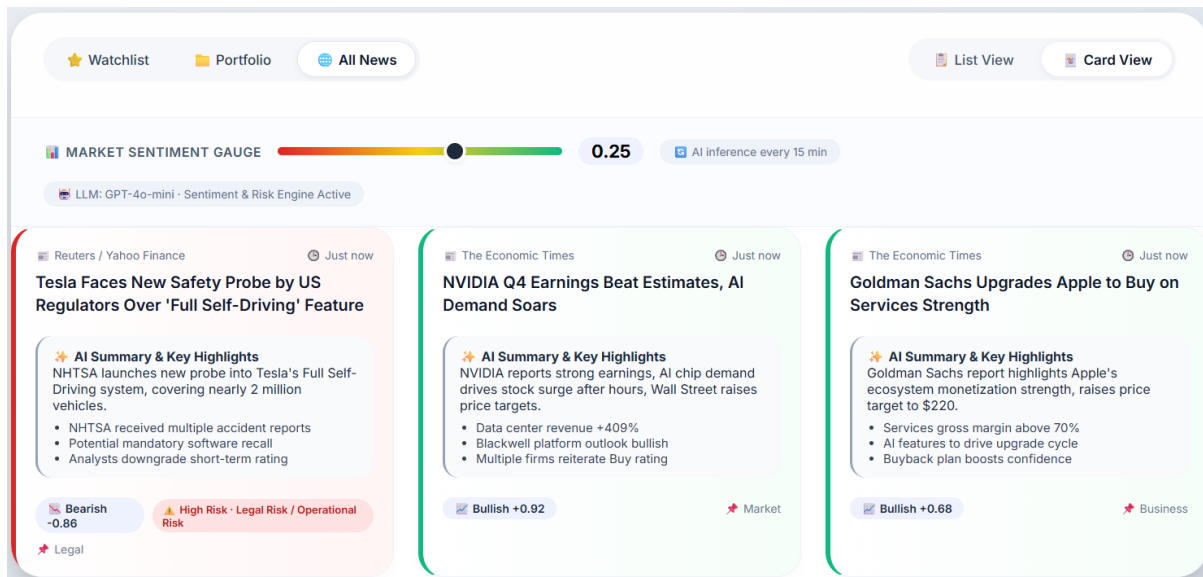
The widget demonstrates **news aggregation**, **AI-style summaries**, **sentiment**, and **risk tagging**, with **list** and **card** layouts.

Completed UI features

- **Aggregation (mock)** — Simulated Yahoo Finance, FUTU, Economic Times-style sources; filters: Watchlist, Portfolio, All.
- **Summaries and highlights** — Short summary and bullets per item.
- **Sentiment gauge** — Score roughly -1.0 (bearish) to +1.0 (bullish); gradient bar; simulated refresh (~15 min).
- **Risk engine (UI)** — Tags such as Legal / Operational / Market risk with High / Medium / Low styling; example narratives (e.g. probe → high risk).
- **List vs card** — List: compact row; detail on hover. Card: full content in a grid; user toggle.
- **Sentiment borders** — e.g. green/red thresholds vs neutral.
- **Sentiment tooltip** — Short “reason” popover on hover.
- **Responsive** behaviour and periodic mock updates for demo purposes.

Next steps

- Live news APIs; backend **LLM** pipeline (e.g. GPT-4o-mini / Claude).
- Persist filters and view mode; **historical** sentiment and event log.



3.6 Context-aware AI chat panel

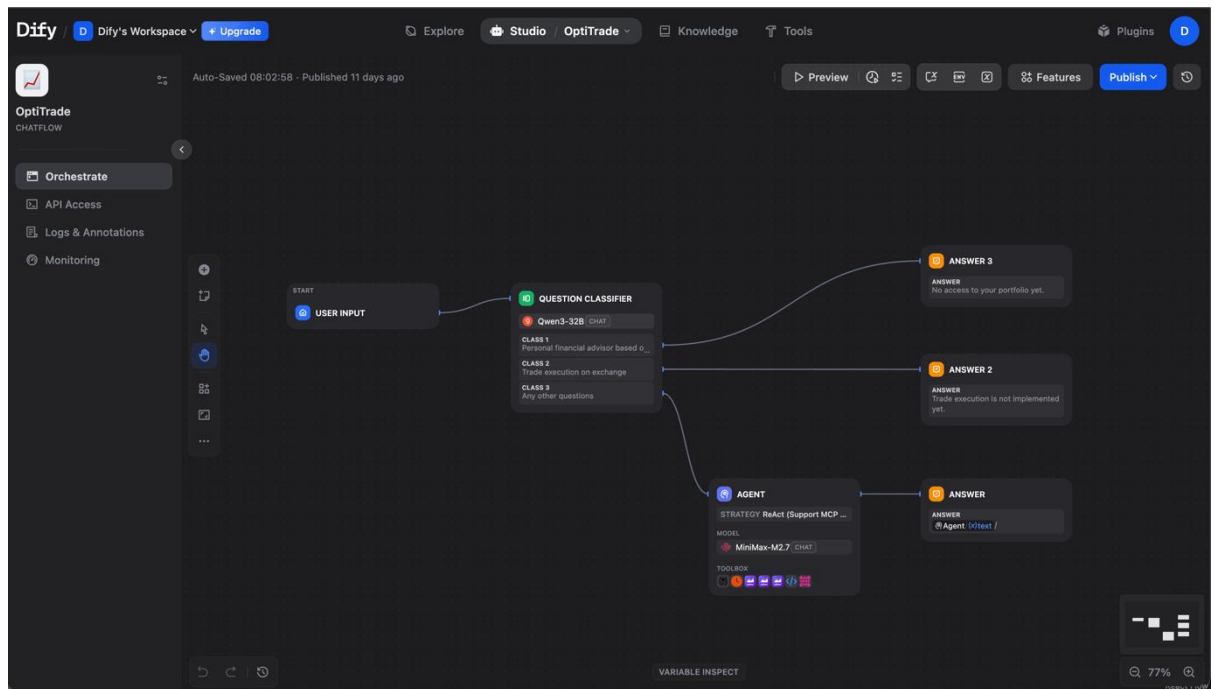
Done

- Basic conversational flow via **Dify**.
- **Embedded** chatbot with **bubble** toggle.

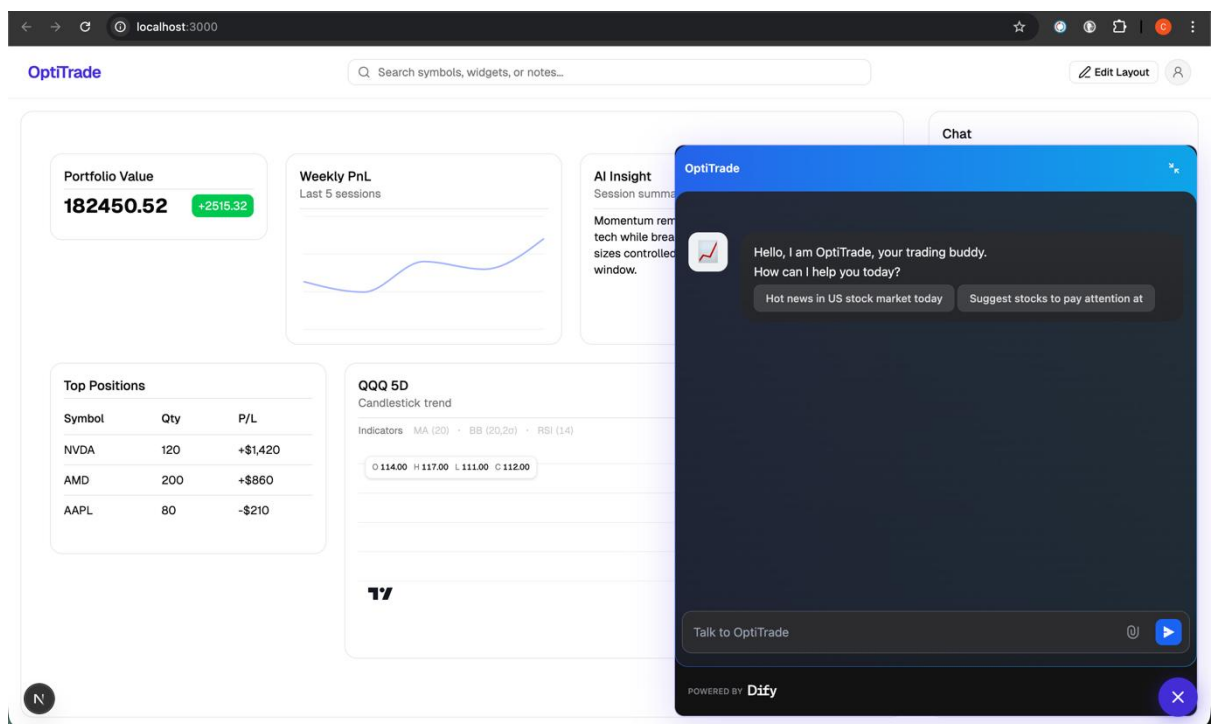
In progress

- Design **question** → **answer** paths for trading/portfolio/news scenarios.
- Implement in **Dify** and mirror/compare in **LangChain**.
- **Context injection** from widgets (portfolio, news, chart selection).
- **Image upload** support in the chat flow.

1. Completed a basic chat bot flow using Dify



2. Embedded the Dify chatbot into the app with a bubble toggle

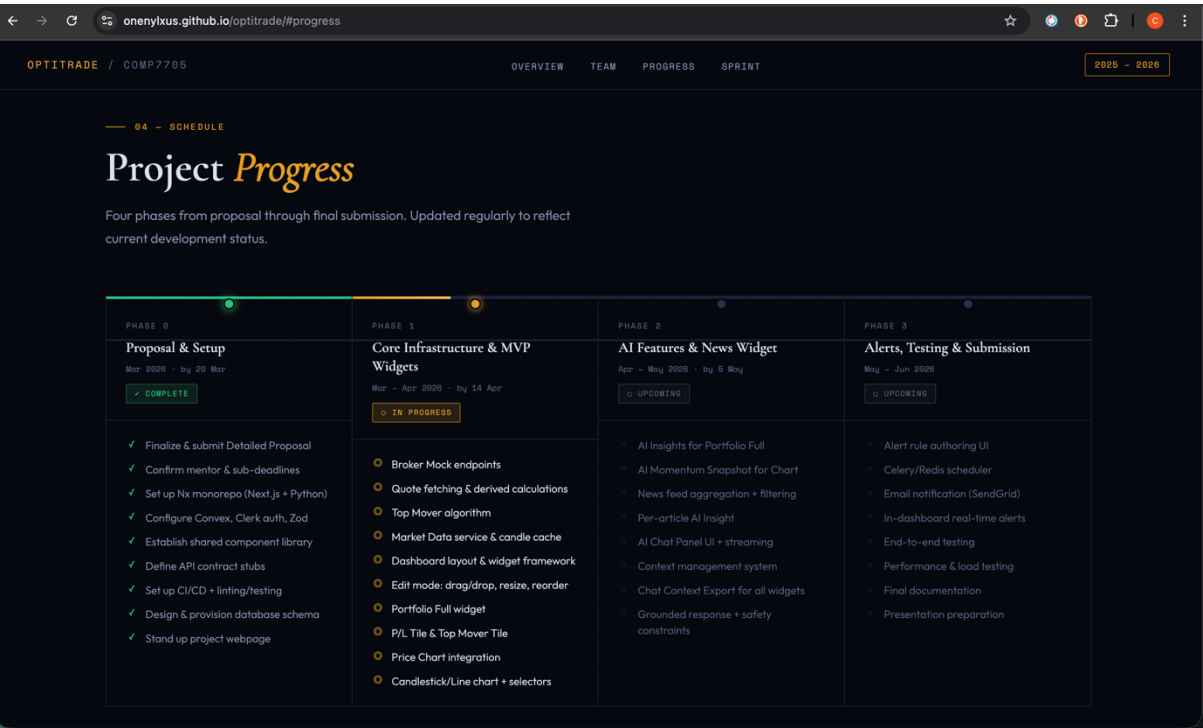


3.7 Project web page

A **public project page** has been published to document the product concept, feature overview, and demo entry point for reviewers and users:

URL: <https://onenylxus.github.io/optitrade/>

The page supports the final-year presentation and external sharing without requiring local installation.



4.0 Challenges encountered and mitigation plan

4.1 Administrative / process

- **Assessment or deliverable claiming (if applicable)** — Confirm with the course or supervisor **which artifacts** (report chapters, demo video, repository tags) map to official milestones, and **deadlines** for submission/approval. *Mitigation*: one checklist aligned with the official brief; single owner to update status weekly.

4.2 News pipeline and AI

Challenge	Severity	Mitigation plan
Latency — polling multiple APIs (Yahoo, FUTU, ET) every ~15 min may be slow	High	Parallel fetches; Redis cache (e.g. TTL 10 min); keyword pre-filter for high-impact items
LLM cost and speed — large batches of items	Medium	Batched inference; model cascade (smaller model for low-impact items); cache entity-level sentiment where stable
List-view discoverability — hover hidden on touch	Low	First-run hint; tap-to-expand on mobile; default card view for new users
API limits / outages	High	Exponential backoff (e.g. max 3 retries); fallback sources;

		serve stale cache when needed; health checks
--	--	---

5.0 Updated plan and next steps

5.1 News workstream (indicative 4–5 weeks)

Phase	Focus	Duration
Phase 1	Real API integration + LLM backend	Week 1
Phase 2	Front-end polish: persistence, loading, live updates	Week 2
Phase 3	Risk engine: high-impact detection, richer risk types	Week 3
Phase 4	Testing (unit, integration, UAT) and performance	Week 4
Phase 5	Deployment, monitoring, feedback	Week 5

5.2 Broader development — infrastructure and engineering

To support the features above, the following **infrastructure and engineering** items are in scope (completed or planned, aligned with the current prototype):

- **Source control and collaboration** — Git repository (e.g. GitHub), branch strategy for features vs demo-stable tags for assessment.
- **Build and quality** — Package manager and scripts for `dev` / `build` / `lint` / `test`; keep dependencies pinned where stability matters for demo week.
- **Front-end hosting** — Static hosting for the **marketing/landing** site (GitHub Pages); separate consideration for any **API** or **LLM proxy** (if not serverless).
- **Secrets and configuration** — No real IBKR or API keys in the repo; use environment variables and documented `.env.example`; plan encryption or vault storage for production-like demos.
- **Observability** — Basic logging for API failures and LLM timeouts; user-visible error states on widgets (news/chart/portfolio).
- **Documentation** — README with setup, feature list, and known limitations (mock vs live data) for examiners and future maintainers.